

CBS3D++_SI

Sunbird HPC Installation, Build and Execution Runbook

Document ID	CBS3D-SUNBIRD-RUN-001
Revision	0.1
Issue date	3 July 2026
Status	Controlled Draft - Phase 1
Owner	Prajwal Bharadwaj
Cluster login	<code>s.2337862@sunbird.swansea.ac.uk</code>
Scratch root	<code>/scratch/s.2337862</code>
Expected solver root	<code>/scratch/s.2337862/CBS3D++_SI</code>
Current development gate	PETSc DMplex Phase 1 mesh distribution

Important

This is a living controlled document. Do not change compiler, MPI, PETSc, partitioner, build flags, source revision, or Slurm resources without recording the change in the document register and retaining the corresponding job output.

Contents

1 Document control	1
1.1 Document register	1
1.2 Revision history	1
1.3 Mandatory run record	2
2 Purpose and current scope	2
3 Fixed identity and directory policy	3
4 Master checklist	3
5 Login and first inspection	4
6 Run the controlled preflight	4
7 Module and package discovery	4
7.1 Start from a clean module environment	4
7.2 Create the controlled module file	5
7.3 Compiler and C++20 check	5
7.4 OpenMP check	6
7.5 MPI wrapper check	6
7.6 PETSc environment check	7
8 PETSc fallback installation	7
9 Project inventory and backup	8
10 Transfer the Phase 1 package	9
11 Controlled Phase 1 installation	9
12 Build procedures	9
12.1 Display the selected build configuration	9
12.2 Build Phase 1	10
12.3 Build the current serial/OpenMP reference solver	10
12.4 MPI bootstrap build	10
12.5 CMake alternative for Phase 1	11
13 Case-file verification	11
14 Slurm Phase 1 acceptance test	12

14.1 Prepare directories and script	12
14.2 Submit	12
14.3 Acceptance criteria	12
 15 Large Phase 1 blanket test	 13
 16 Current serial/OpenMP solver run	 13
 17 Troubleshooting matrix	 14
 18 Phase 1 close-out and revision gate	 15
 A Controlled preflight script	 16
 B Controlled module-file template	 18
 C Phase 1 installation script	 18
 D Phase 1 build script	 19
 E Phase 1 Slurm job	 20
 F Current serial/OpenMP Slurm job	 21
 G Conditional PETSc source-build script	 22
 H Source register	 23

1 Document control

1.1 Document register

Document ID	Controlled file	Rev.	Status	Purpose
CBS3D-SUNBIRD-RUN-001	CBS3D_Sunbird_Runbook.tex	0.1	Controlled Draft	Primary installation, build, execution and acceptance runbook.
CBS3D-SUNBIRD-RUN-001-PDF	CBS3D_Sunbird_Runbook.pdf	0.1	Controlled Draft	Rendered copy of this runbook.
CBS3D-SUNBIRD-ENV-001	sunbird_modules.sh	Site	Working controlled file	Exact compiler, MPI and PETSc modules used for reproducible builds.
CBS3D-SUNBIRD-CHK-001	sunbird_preflight.sh	0.1	Issued	Cluster, toolchain, MPI and PETSc checks.
CBS3D-SUNBIRD-INS-001	install_phase1.sh	0.1	Issued	Backup and controlled installation of Phase 1 files.
CBS3D-SUNBIRD-BLD-001	build_phase1.sh	0.1	Issued	Phase 1 Make build.
CBS3D-SUNBIRD-JOB-001	phase1_test.slurm	0.1	Issued	One-, two- and four-rank Phase 1 acceptance job.
CBS3D-SUNBIRD-JOB-002	serial_openmp.slurm	0.1	Issued	Current serial/OpenMP or serial PETSc solver job.
CBS3D-SUNBIRD-PETSC-001	build_petsc_fallback.sh	0.1	Conditional	PETSc source build only when no compatible site module exists.
CBS3D-PHASE1-PKG-001	CBS3D_Phase1_PETSc_DMPlex.zip	0.1	Issued	Phase 1 PETSc DMPlex source and build package.

The machine-readable register is supplied as `DOCUMENT_REGISTER.csv`. Update both the CSV and this table whenever a controlled file is revised.

1.2 Revision history

Rev.	Date	Change	Approval state
0.1	3 July 2026	Initial controlled Sunbird runbook. Covers preflight, toolchain discovery, PETSc verification, Phase 1 installation/build/test, the current serial/OpenMP solver, and a PETSc source-build fallback.	Controlled Draft

1.3 Mandatory run record

For every build and run, create a record containing:

- date and time;
- Git commit or source archive checksum;
- loaded modules and versions;
- compiler, MPI and PETSc versions;
- PETSC_DIR and PETSC_ARCH;
- build command and full compiler output;
- case name and input-file checksums;
- Slurm job ID, account, partition, nodes, MPI ranks and OpenMP threads;
- standard output, standard error and acceptance result.

2 Purpose and current scope

This runbook defines a reproducible route from a clean Sunbird login session to:

1. checking the Sunbird environment and storage;
2. discovering and recording compatible compiler, MPI, PETSc and build tools;
3. safely installing the CBS3D++_SI Phase 1 DMPLex files;
4. building and testing the Phase 1 distributed mesh executable;
5. building and running the current validated serial/OpenMP or serial PETSc solver;
6. submitting jobs through Slurm and recording evidence.

Important

The current executable `cbs3d_mesh_phase1` validates distributed mesh creation, partitioning, material labels, boundary labels, owned points and ghost points. It does not run CBS Steps 1–4. The current MPI-enabled main solver is an MPI bootstrap in which rank zero still performs the complete numerical calculation. Distributed numerical acceleration is not claimed until later phases pass their validation gates.

3 Fixed identity and directory policy

Item	Controlled value
SSH login	s.2337862@sunbird.swansea.ac.uk
Login command	ssh s.2337862@sunbird.swansea.ac.uk
Scratch root	/scratch/s.2337862
Expected solver root	/scratch/s.2337862/CBS3D++_SI
Package staging	/scratch/s.2337862/packages
Source backups	/scratch/s.2337862/backups
Solver logs	/scratch/s.2337862/CBS3D++_SI/logs
Module environment file	/scratch/s.2337862/CBS3D++_SI/sunbird_modules.sh

Sunbird compute jobs are managed by Slurm. The system portal currently describes 40 CPU cores and 384 GB RAM per standard node, but every submission must verify the live configuration using `sinfo`, `scontrol` and the allocated node. Scratch is high-performance temporary storage and should not be treated as backed-up archival storage.

Stop condition

Do not run the large CBS solver or a PETSc source build directly on a login node. Compile modest project changes on the login node only when permitted by site policy; perform heavy compilation and all numerical tests in a Slurm allocation. Retain source and results outside scratch when they are valuable.

4 Master checklist

No.	Gate	Record
1	☐ Login identity is s.2337862 and scratch path is accessible.	Date/initials
2	☐ Slurm account and live partition names are recorded.	Account
3	☐ Exact compiler, MPI, PETSc and CMake modules are recorded.	Module set ID
4	☐ C++20, OpenMP, MPI wrapper and PETSc preflight checks pass.	Log file
5	☐ PETSc reports required features: HYPRE and at least one suitable partitioner.	Feature record
6	☐ Existing CBS source is backed up before Phase 1 installation.	Backup path
7	☐ Phase 1 files are installed and checksummed.	Checksum file
8	☐ Phase 1 executable builds with zero compile errors.	Build log
9	☐ One-rank Phase 1 test passes.	Job ID
10	☐ Two-rank simple-partitioner test passes.	Job ID
11	☐ Four-rank automatic-partitioner test passes.	Job ID
12	☐ Global cells, vertices, boundary faces and material totals match.	PASS evidence
13	☐ Current serial/OpenMP reference case still runs correctly.	Regression job ID
14	☐ Register and revision history are updated.	New revision

5 Login and first inspection

From a local terminal:

```
ssh s.2337862@sunbird.swansea.ac.uk
```

On Sunbird:

```
whoami
hostname
pwd
cd /scratch/s.2337862
pwd
ls -lah
df -h /scratch/s.2337862
myquota
```

The expected working-directory output is:

```
/scratch/s.2337862
```

Inspect Slurm:

```
sinfo -s
squeue -u s.2337862
sacctmgr -n -P show assoc where user=s.2337862 \
    format=Account,Partition,QOS
```

Select the approved project account from the association output and record it:

```
export CBS_ACCOUNT='the-approved-account-returned-by-sacctmgr'
printf 'CBS_ACCOUNT=%s\n' "$CBS_ACCOUNT"
```

Do not use a training account, reservation or project code copied from generic documentation.

6 Run the controlled preflight

Copy `sunbird_preflight.sh` into the solver directory and run:

```
cd /scratch/s.2337862/CBS3D++_SI
chmod u+x sunbird_preflight.sh

./sunbird_preflight.sh \
    2>&1 | tee logs/preflight_$(date +%Y%m%d_%H%M%S).log
```

The script is reproduced in Appendix A. Review every MISSING, ERROR and WARNING before proceeding.

7 Module and package discovery

7.1 Start from a clean module environment

```
module purge
module list
module avail 2>&1 | less
```

Search without guessing names:

```
module spider GCC
module spider OpenMPI
module spider MPI
module spider PETSc
module spider CMake
```

If module `spider` is unavailable, use:

```
module avail 2>&1 | grep -Ei 'gcc|gnu|openmpi|intel|mpi|petsc|cmake'
```

Select one compatible software family. The compiler used by `mpicxx` must be ABI-compatible with the compiler used to build PETSc.

Important

Do not mix a PETSc module built with one compiler/MPI family with a different loaded MPI wrapper. This can compile successfully and then fail at link time or during MPI initialisation.

7.2 Create the controlled module file

Copy the supplied template:

```
cd /scratch/s.2337862/CBS3D++_SI

cp sunbird_modules.sh.template sunbird_modules.sh
chmod u+x sunbird_modules.sh
nano sunbird_modules.sh
```

Insert the exact module commands reported by Sunbird, then test:

```
source /scratch/s.2337862/CBS3D++_SI/sunbird_modules.sh

module list
which g++
which mpicxx
which mpirun
echo "$PETSC_DIR"
echo "${PETSC_ARCH:-unset}"
```

Record the exact output in `DOCUMENT_REGISTER.csv`.

7.3 Compiler and C++20 check

```
g++ --version
mpicxx --version
cmake --version
make --version

cat > /tmp/cbs_cxx20_test.cpp <<'EOF'
#include <span>
#include <vector>
int main() {
    std::vector<int> values{1, 2, 3};
    std::span<int> view(values);
```

```

    return view.size() == 3 ? 0 : 1;
}
EOF

g++ -std=c++20 /tmp/cbs_cxx20_test.cpp -o /tmp/cbs_cxx20_test
/tmp/cbs_cxx20_test
echo "C++20 return code=$?"

```

A zero return code is required.

7.4 OpenMP check

```

cat > /tmp/cbs_openmp_test.cpp <<'EOF'
#include <omp.h>
#include <iostream>
int main() {
    #pragma omp parallel
    {
        #pragma omp single
        std::cout << "OpenMP threads=" << omp_get_num_threads() << '\n';
    }
}
EOF

g++ -std=c++20 -fopenmp /tmp/cbs_openmp_test.cpp \
    -o /tmp/cbs_openmp_test

OMP_NUM_THREADS=4 /tmp/cbs_openmp_test

```

Expected output:

```
OpenMP threads=4
```

7.5 MPI wrapper check

```

which mpicc
which mpicxx
which mpirun

mpicxx --showme:command 2>/dev/null || mpicxx -show
mpirun --version

```

Compile a minimal MPI test:

```

cat > /tmp/cbs_mpi_test.cpp <<'EOF'
#include <mpi.h>
#include <iostream>
int main(int argc, char** argv) {
    MPI_Init(&argc, &argv);
    int rank = 0;
    int size = 1;
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    std::cout << "rank=" << rank << " size=" << size << '\n';
    MPI_Finalize();
}
EOF

```

```
mpicxx -std=c++20 /tmp/cbs_mpi_test.cpp -o /tmp/cbs_mpi_test
```

Run the executable only in a Slurm allocation, for example:

```
salloc --account="$CBS_ACCOUNT" --partition=compute \
  --nodes=1 --ntasks=2 --time=00:10:00

srun --ntasks=2 /tmp/cbs_mpi_test
exit
```

Both ranks must report `size=2`.

7.6 PETSc environment check

```
echo "PETSC_DIR=$PETSC_DIR"
echo "PETSC_ARCH=${PETSC_ARCH:-unset}"

if [[ -n "${PETSC_ARCH:-}" ]]; then
  test -f "$PETSC_DIR/$PETSC_ARCH/include/petscconf.h"
  PETSC_CONF="$PETSC_DIR/$PETSC_ARCH/include/petscconf.h"
else
  test -f "$PETSC_DIR/include/petscconf.h"
  PETSC_CONF="$PETSC_DIR/include/petscconf.h"
fi

grep -E 'PETSC_HAVE_(HYPRE|PARMETIS|PTSCOTCH|METIS)' \
  "$PETSC_CONF" || true
```

The preferred Phase 1 configuration contains:

- PETSC_HAVE_PARMETIS, or PETSC_HAVE_PTSCOTCH;
- PETSC_HAVE_HYPRE for the later distributed pressure phase.

The Phase 1 code can fall back to PETSc's simple partitioner, but the production target is ParMETIS or PT-Scotch.

Inspect linkage:

```
find "$PETSC_DIR" -path '*/lib/libpetsc.so' -print
pkg-config --modversion PETSc 2>/dev/null || true
pkg-config --cflags PETSc 2>/dev/null || true
pkg-config --libs PETSc 2>/dev/null || true
```

8 PETSc fallback installation

Use the site PETSc module when it is compatible. Build PETSc from source only when no suitable module exists or when the site build lacks required packages.

Stop condition

The PETSc fallback build downloads and compiles PETSc, BLAS/LAPACK, HYPRE, METIS, ParMETIS and PT-Scotch. Run it in an allocated compute session and retain the full configure and check logs. Do not mix the resulting installation with a different MPI module later.

Allocate resources:

```
salloc --account="$CBS_ACCOUNT" --partition=compute \
  --nodes=1 --ntasks=1 --cpus-per-task=16 \
  --mem=64G --time=04:00:00

cd /scratch/s.2337862/CBS3D++_SI
source sunbird_modules.sh
./build_petsc_fallback.sh 2>&1 | tee \
  logs/petsc_build_$(date +%Y%m%d_%H%M%S).log

exit
```

After installation, set:

```
export PETSC_DIR=/scratch/s.2337862/software/petsc-release
unset PETSC_ARCH
```

and record these lines in the controlled `sunbird_modules.sh`.

9 Project inventory and backup

Set and verify the exact project path:

```
export CBS_ROOT=/scratch/s.2337862/CBS3D++_SI

test -d "$CBS_ROOT"
cd "$CBS_ROOT"
pwd

test -f Makefile
test -f CMakeLists.txt
test -d include/cbs
test -d src
```

Inventory the source:

```
find include src -type f | sort > \
  logs/source_inventory_$(date +%Y%m%d_%H%M%S).txt

find . -maxdepth 2 -type f \
  \( -name '*.plt' -o -name '*.bco' -o -name '*.par' \
    -o -name '*.material' -o -name '*.matprop' \) \
  -print | sort
```

Create a source backup before Phase 1 installation:

```
mkdir -p /scratch/s.2337862/backups

stamp=$(date +%Y%m%d_%H%M%S)

tar -czf \
  "/scratch/s.2337862/backups/CBS3D++_SI_source_${stamp}.tar.gz" \
  -C "$CBS_ROOT" Makefile CMakeLists.txt include src

sha256sum \
  "/scratch/s.2337862/backups/CBS3D++_SI_source_${stamp}.tar.gz"
```

10 Transfer the Phase 1 package

From Windows PowerShell:

```
scp "$env:USERPROFILE\Downloads\CBS3D_Phase1_PETSc_DMPLex.zip" '
s.2337862@sunbird.swansea.ac.uk:/scratch/s.2337862/
```

On Sunbird:

```
cd /scratch/s.2337862

test -f CBS3D_Phase1_PETSc_DMPLex.zip
sha256sum CBS3D_Phase1_PETSc_DMPLex.zip
unzip -l CBS3D_Phase1_PETSc_DMPLex.zip
```

Retain the checksum in the run record.

11 Controlled Phase 1 installation

Copy the supplied installation script to the project and execute it:

```
cd /scratch/s.2337862/CBS3D++_SI
chmod u+x install_phase1.sh

./install_phase1.sh 2>&1 | tee \
  logs/phase1_install_$(date +%Y%m%d_%H%M%S).log
```

Verify the installed files:

```
test -f include/cbs/parallel/PetscMeshDistributor.hpp
test -f src/parallel/PetscMeshDistributor.cpp
test -f src/tests/mesh_distribution_test.cpp
test -f PHASE1_TESTING.md

sha256sum \
  include/cbs/parallel/PetscMeshDistributor.hpp \
  src/parallel/PetscMeshDistributor.cpp \
  src/tests/mesh_distribution_test.cpp \
  Makefile CMakeLists.txt \
  > logs/phase1_source_checksums.txt
```

12 Build procedures

12.1 Display the selected build configuration

```
cd /scratch/s.2337862/CBS3D++_SI
source sunbird_modules.sh

make print-config \
  USE_MPI=1 \
  USE_PETSC=1 \
  PETSC_DIR="$PETSC_DIR" \
  ${PETSC_ARCH:+PETSC_ARCH="$PETSC_ARCH"}
```

Check that:

- CXX resolves to the intended MPI C++ wrapper;
- USE_MPI=1;
- USE_PETSC=1;
- PETSC_DIR and, when applicable, PETSC_ARCH are correct.

12.2 Build Phase 1

```
cd /scratch/s.2337862/CBS3D++_SI
chmod u+x build_phase1.sh

./build_phase1.sh 2>&1 | tee \
  logs/phase1_build_$(date +%Y%m%d_%H%M%S).log
```

Required artifact:

```
test -x /scratch/s.2337862/CBS3D++_SI/cbs3d_mesh_phase1
```

12.3 Build the current serial/OpenMP reference solver

Without PETSc:

```
make clean
make USE_OPENMP=1
```

Expected executable:

```
cbs3dpp_si
```

With the current serial PETSc pressure path:

```
make clean

if [[ -n "${PETSC_ARCH:-}" ]]; then
  make USE_OPENMP=1 USE_PETSC=1 \
    PETSC_DIR="$PETSC_DIR" PETSC_ARCH="$PETSC_ARCH"
else
  make USE_OPENMP=1 USE_PETSC=1 \
    PETSC_DIR="$PETSC_DIR"
fi
```

Expected executable:

```
cbs3dpp_si_petsc
```

12.4 MPI bootstrap build

```
make clean

if [[ -n "${PETSC_ARCH:-}" ]]; then
  make USE_MPI=1 USE_OPENMP=1 USE_PETSC=1 \
    PETSC_DIR="$PETSC_DIR" PETSC_ARCH="$PETSC_ARCH"
else
  make USE_MPI=1 USE_OPENMP=1 USE_PETSC=1 \
    PETSC_DIR="$PETSC_DIR"
fi
```

Expected executable:

```
cbs3dpp_si_mpi_petsc
```

Important

This MPI bootstrap is not yet the accelerated distributed solver. All ranks initialise correctly, but rank zero still executes the complete CBS numerical path. Use `cbs3d_mesh_phase1` for the Phase 1 distributed mesh tests.

12.5 CMake alternative for Phase 1

```
rm -rf build-phase1

cmake -S . -B build-phase1 \
  -DCBS3D_ENABLE_OPENMP=ON \
  -DCBS3D_ENABLE_MPI=ON \
  -DCBS3D_ENABLE_PETSC=ON \
  -DCBS3D_BUILD_MESH_PHASE1=ON

cmake --build build-phase1 --parallel
```

The Make workflow is the controlled baseline for Phase 1. Use CMake as a secondary consistency check.

13 Case-file verification

Before submission, verify the case files expected by the current MeshIO path. For a case called `LidDrivenCavity3D`:

```
cd /scratch/s.2337862/CBS3D++_SI

find . -maxdepth 3 -type f \
  -name 'LidDrivenCavity3D*' -print | sort
```

For the CHT blanket case:

```
find . -maxdepth 3 -type f \
  -name 'blanket*' -print | sort
```

Depending on the case, the controlled input set may include:

```
<case>.plt
<case>.bco
<case>.par
<case>.material
<case>.matprop
```

Record checksums:

```
sha256sum <case>.plt <case>.bco <case>.par \
  2>/dev/null | tee logs/<case>_input_checksums.txt
```

14 Slurm Phase 1 acceptance test

14.1 Prepare directories and script

```
cd /scratch/s.2337862/CBS3D++_SI
mkdir -p logs

chmod u+x phase1_test.slurm
```

Confirm the live partition name:

```
sinfo -s
```

The issued script uses `compute`. Change it only if `sinfo -s` shows that the valid partition name has changed, and record the revision.

14.2 Submit

For the small reference case:

```
export CBS_CASE=LidDrivenCavity3D

sbatch \
  --account="$CBS_ACCOUNT" \
  --export=ALL,CBS_CASE="$CBS_CASE" \
  phase1_test.slurm
```

Record the returned job ID.

Monitor:

```
squeue -u s.2337862
scontrol show job <job-id>
tail -f logs/phase1.<job-id>.out
```

After completion:

```
sacct -j <job-id> \
  --format=JobID,JobName,Partition,AllocCPUS,Elapsed,State,ExitCode,MaxRSS

seff <job-id> 2>/dev/null || true

cat logs/phase1.<job-id>.err
```

14.3 Acceptance criteria

All three subtests must complete:

1. one rank with the simple partitioner;
2. two ranks with the simple partitioner;
3. four ranks with automatic partitioner selection.

Required output for every subtest:

```
Global count verification      : PASS
PHASE 1 RESULT: PASS
```

The following equalities must hold:

```
Reconstructed owned cells      = Expected global cells
Reconstructed owned vertices    = Expected global vertices
Reconstructed boundary faces    = Expected CBS boundary faces
Fluid cells + Solid cells      = Expected global cells
```

The run record must also contain per-rank:

- owned and ghost cells;
- owned and ghost vertices;
- labelled boundary faces;
- fluid and solid cell counts;
- neighbour-rank counts;
- selected partitioner;
- cell and vertex imbalance.

Stop condition

Do not proceed to distributed field sections or numerical assembly if any global count is wrong, a material or boundary label is missing, or any rank terminates abnormally.

15 Large Phase 1 blanket test

Run only after the small case passes.

A four-rank job can be submitted by reusing the issued script:

```
export CBS_CASE=blanket

sbatch \
  --account="$CBS_ACCOUNT" \
  --export=ALL,CBS_CASE="$CBS_CASE" \
  phase1_test.slurm
```

The current Phase 1 design still reads the complete global mesh on rank zero before DMPLex distribution. Rank-zero memory must therefore be monitored using Slurm accounting. This limitation is intentional for Phase 1 and must be removed in a later scalable I/O phase.

16 Current serial/OpenMP solver run

The current validated reference solver remains important because distributed results must eventually reproduce it.

Build the intended reference executable, then submit:

```
export CBS_CASE=LidDrivenCavity3D
export CBS_EXECUTABLE=cbs3dpp_si_petsc
```

```

sbatch \
  --account="$CBS_ACCOUNT" \
  --export=ALL,CBS_CASE="$CBS_CASE",CBS_EXECUTABLE="$CBS_EXECUTABLE" \
  serial_openmp.slurm

```

For a larger case:

```

export CBS_CASE=blanket
export CBS_EXECUTABLE=cbs3dpp_si_petsc

sbatch \
  --account="$CBS_ACCOUNT" \
  --export=ALL,CBS_CASE="$CBS_CASE",CBS_EXECUTABLE="$CBS_EXECUTABLE" \
  serial_openmp.slurm

```

Review:

- solver banner and parsed input dimensions;
- material and boundary audits;
- pressure KSP/CG convergence;
- CBS residual history;
- VTU/PVD output;
- profiler breakdown;
- job memory and CPU efficiency.

17 Troubleshooting matrix

Symptom	Likely cause	Controlled response
mpicxx: command not found	MPI module not loaded or wrong module family.	Run <code>module purge</code> ; repeat module discovery; source the controlled module file; verify which <code>mpicxx</code> .
PETSC_DIR is not set	PETSc module did not export its root or the local PETSc installation was not recorded.	Inspect <code>module show</code> ; set the correct installation root in <code>sunbird_modules.sh</code> ; do not guess.
petsc.h: No such file	Wrong PETSc include root or missing PETSC_ARCH.	Locate <code>petscconf.h</code> ; verify whether the installation is prefix-based or source-tree based.
libpetsc.so: cannot open	Runtime linker path does not include the PETSc library.	Re-source the PETSc module; inspect <code>LD_LIBRARY_PATH</code> ; verify Makefile rpath and <code>PETSC_DIR</code> .
MPI initialisation failure	PETSc and executable use different MPI implementations.	Compare <code>mpicxx -show</code> , PETSc configure information and <code>ldd libpetsc.so</code> ; rebuild consistently.
Unknown partitioner parmetis	PETSc lacks ParMETIS.	Use <code>-petscpartitioner_type simple</code> for the diagnostic test, then use a PETSc build with ParMETIS or PT-Scotch for production.

Symptom	Likely cause	Controlled response
Boundary count FAIL	Boundary triangle did not map uniquely to a DMPlex face, or BC labels were not preserved.	Stop Phase 1; retain the full job output; compare <code>iside</code> connectivity with the global vertex IDs.
Vertex count FAIL	Incorrect owner/ghost classification or global vertex label migration.	Stop; inspect point SF and global vertex labels using <code>-cbs_mesh_view</code> .
Rank-zero out of memory	Phase 1 root mesh construction still holds the full global mesh.	Use a smaller case for validation; increase memory only for diagnosis; implement distributed mesh input in a later phase.
Poor OpenMP performance	Oversubscription, poor affinity or too many MPI ranks per node.	Set <code>OMP_NUM_THREADS</code> , <code>OMP_PROC_BIND=close</code> , <code>OMP_PLACES=cores</code> ; ensure ranks times threads do not exceed allocated CPUs.
Job remains pending	Account, partition, time, memory or resource request cannot currently be scheduled.	Run <code>squeue</code> , <code>scontrol show job</code> and inspect the pending reason; do not repeatedly resubmit identical jobs.

18 Phase 1 close-out and revision gate

Phase 1 may be closed only when:

- the small one-, two- and four-rank tests pass;
- all global entity and label totals match;
- the selected partitioner and PETSc features are recorded;
- the current serial/OpenMP reference solver still passes its regression case;
- the job outputs and source checksums are retained;
- this document and `DOCUMENT_REGISTER.csv` are revised.

The next controlled revision should add Phase 2 procedures for:

- velocity, pressure and temperature `PetscSection` layouts;
- local and global PETSc vectors;
- additive local-to-global finite-element residual assembly;
- owner-to-ghost field refresh;
- serial-versus-distributed assembly comparisons.

A Controlled preflight script

Listing 1: sunbird_preflight.sh

```
#!/bin/bash --login
set -u

LOGIN_ID="s.2337862"
SCRATCH_ROOT="/scratch/s.2337862"
CBS_ROOT="${SCRATCH_ROOT}/CBS3D++_SI"

section() {
    printf '\n=====\\n'
    printf '%s\\n' "$1"
    printf '=====\\n'
}

run_optional() {
    printf '\\n$ %s\\n' "$*"
    "$@" 2>&1 || true
}

section "Identity and storage"
printf 'USER=%s\\n' "${USER:-unset}"
printf 'HOSTNAME=%s\\n' "$(hostname)"
printf 'PWD=%s\\n' "$(pwd)"
printf 'Expected login ID=%s\\n' "$LOGIN_ID"
printf 'Expected scratch root=%s\\n' "$SCRATCH_ROOT"
printf 'Expected solver root=%s\\n' "$CBS_ROOT"

if [[ "${USER:-}" != "$LOGIN_ID" ]]; then
    printf 'WARNING: current USER is not %s\\n' "$LOGIN_ID"
fi

if [[ ! -d "$SCRATCH_ROOT" ]]; then
    printf 'ERROR: %s does not exist or is not accessible.\\n' "$SCRATCH_ROOT"
fi

run_optional df -h "$SCRATCH_ROOT"
run_optional myquota

section "Slurm"
run_optional sinfo -s
run_optional squeue -u "$LOGIN_ID"
run_optional sacctmgr -n -P show assoc where user="$LOGIN_ID" format=Account,Partition,
    QOS

section "Loaded modules"
run_optional module list

section "Build tools"
for command_name in bash make cmake gcc g++ mpicc mpicxx mpirun srun pkg-config git
    unzip rsync; do
    if command -v "$command_name" >/dev/null 2>&1; then
        printf '%-12s %s\\n' "$command_name" "$(command -v "$command_name")"
    else
        printf '%-12s MISSING\\n' "$command_name"
    fi
done
```

```

run_optional g++ --version
run_optional cmake --version
run_optional make --version
run_optional mpicxx --version
run_optional mpirun --version

section "MPI wrapper"
if command -v mpicxx >/dev/null 2>&1; then
    mpicxx --showme:command 2>/dev/null || mpicxx -show 2>/dev/null || true
    mpicxx --showme:compile 2>/dev/null || true
    mpicxx --showme:link 2>/dev/null || true
fi

section "PETSc environment"
printf 'PETSC_DIR=%s\n' "${PETSC_DIR:-unset}"
printf 'PETSC_ARCH=%s\n' "${PETSC_ARCH:-unset}"

PETSC_CONF=""
if [[ -n "${PETSC_DIR:-}" ]]; then
    if [[ -n "${PETSC_ARCH:-}" ]] && -f "${PETSC_DIR}/${PETSC_ARCH}/include/petscconf.h"
    ]]; then
        PETSC_CONF="${PETSC_DIR}/${PETSC_ARCH}/include/petscconf.h"
    elif [[ -f "${PETSC_DIR}/include/petscconf.h" ]]; then
        PETSC_CONF="${PETSC_DIR}/include/petscconf.h"
    else
        PETSC_CONF="$(find "$PETSC_DIR" -path '*/include/petscconf.h' -print -quit 2>/
dev/null || true)"
    fi
fi

if [[ -n "$PETSC_CONF" ]] && -f "$PETSC_CONF" ]]; then
    printf 'PETSc configuration header=%s\n' "$PETSC_CONF"
    grep -E 'PETSC_HAVE_(HYPRE|PARMETIS|PTSCOTCH|METIS)' "$PETSC_CONF" || true
else
    printf 'PETSc configuration header not found.\n'
fi

if command -v pkg-config >/dev/null 2>&1; then
    pkg-config --modversion PETSc 2>/dev/null || true
    pkg-config --cflags PETSc 2>/dev/null || true
    pkg-config --libs PETSc 2>/dev/null || true
fi

section "CBS3D++_SI project"
if [[ -d "$CBS_ROOT" ]]; then
    printf 'Solver directory exists.\n'
    find "$CBS_ROOT" -maxdepth 2 -type f \
        \( -name Makefile -o -name CMakeLists.txt -o -name '*.plt' -o -name '*.par' \) \
        -print | sort | head -100
else
    printf 'ERROR: solver directory not found at %s\n' "$CBS_ROOT"
fi

section "Preflight finished"
printf 'Save this output in the document register before changing modules or source
files.\n'

```

B Controlled module-file template

Listing 2: sunbird_modules.sh.template

```
#!/usr/bin/env bash
# CBS3D++_SI controlled Sunbird module environment.
#
# 1. Run:
#     module purge
#     module avail
#     module spider GCC
#     module spider OpenMPI
#     module spider PETSc
#     module spider CMake
#
# 2. Replace the commented examples below with the exact compatible module
#     names reported by Sunbird. Do not guess module names or combine software
#     stacks built with different compiler/MPI families.
#
# 3. Record the selected module names and versions in DOCUMENT_REGISTER.csv.

module purge

# Examples only - replace with exact modules available on Sunbird:
# module load compiler/gnu/<version>
# module load mpi/openmpi/<version>
# module load PETSc/<version>
# module load CMake/<version>

module list

if ! command -v mpicxx >/dev/null 2>&1; then
    echo "ERROR: mpicxx is unavailable after loading modules." >&2
    return 1 2>/dev/null || exit 1
fi

if [[ -z "${PETSC_DIR:-}" ]]; then
    echo "ERROR: PETSC_DIR is not set after loading the PETSc environment." >&2
    return 1 2>/dev/null || exit 1
fi
```

C Phase 1 installation script

Listing 3: install_phase1.sh

```
#!/usr/bin/env bash
set -euo pipefail

SCRATCH_ROOT="/scratch/s.2337862"
CBS_ROOT="${SCRATCH_ROOT}/CBS3D++_SI"
PACKAGE_ZIP="${SCRATCH_ROOT}/CBS3D_Phase1_PETSc_DMPlex.zip"
STAGING_ROOT="${SCRATCH_ROOT}/packages/CBS3D_Phase1_PETSc_DMPlex"
BACKUP_ROOT="${SCRATCH_ROOT}/backups"

timestamp="$(date +%Y%m%d_%H%M%S)"

test -d "$CBS_ROOT"
test -f "$PACKAGE_ZIP"
```

```

mkdir -p "$STAGING_ROOT" "$BACKUP_ROOT"

tar -czf "${BACKUP_ROOT}/CBS3D++_SI_source_${timestamp}.tar.gz" \
  -C "$CBS_ROOT" \
  Makefile \
  CMakeLists.txt \
  include \
  src

rm -rf "${STAGING_ROOT:?}"/*
unzip -q "$PACKAGE_ZIP" -d "$STAGING_ROOT"

install -D -m 0644 \
  "$STAGING_ROOT/include/cbs/parallel/PetscMeshDistributor.hpp" \
  "$CBS_ROOT/include/cbs/parallel/PetscMeshDistributor.hpp"

install -D -m 0644 \
  "$STAGING_ROOT/src/parallel/PetscMeshDistributor.cpp" \
  "$CBS_ROOT/src/parallel/PetscMeshDistributor.cpp"

install -D -m 0644 \
  "$STAGING_ROOT/src/tests/mesh_distribution_test.cpp" \
  "$CBS_ROOT/src/tests/mesh_distribution_test.cpp"

install -m 0644 \
  "$STAGING_ROOT/Makefile" \
  "$CBS_ROOT/Makefile"

install -m 0644 \
  "$STAGING_ROOT/CMakeLists.txt" \
  "$CBS_ROOT/CMakeLists.txt"

install -m 0644 \
  "$STAGING_ROOT/PHASE1_TESTING.md" \
  "$CBS_ROOT/PHASE1_TESTING.md"

echo "Phase 1 files installed."
echo "Source backup: ${BACKUP_ROOT}/CBS3D++_SI_source_${timestamp}.tar.gz"

```

D Phase 1 build script

Listing 4: build_phase1.sh

```

#!/bin/bash --login
set -euo pipefail

SCRATCH_ROOT="/scratch/s.2337862"
CBS_ROOT="${SCRATCH_ROOT}/CBS3D++_SI"
MODULE_FILE="${CBS_ROOT}/sunbird_modules.sh"

if [[ ! -f "$MODULE_FILE" ]]; then
  echo "ERROR: controlled module file not found: $MODULE_FILE" >&2
  exit 1
fi

# shellcheck source=/dev/null
source "$MODULE_FILE"

```

```

cd "$CBS_ROOT"

required_files=(
    "Makefile"
    "include/cbs/parallel/PetscMeshDistributor.hpp"
    "src/parallel/PetscMeshDistributor.cpp"
    "src/tests/mesh_distribution_test.cpp"
)

for required_file in "${required_files[@]"; do
    if [[ ! -f "$required_file" ]]; then
        echo "ERROR: missing $CBS_ROOT/$required_file" >&2
        exit 1
    fi
done

echo "Compiler and library configuration"
make print-config \
    USE_MPI=1 \
    USE_PETSC=1 \
    PETSC_DIR="$PETSC_DIR" \
    ${PETSC_ARCH:+PETSC_ARCH="$PETSC_ARCH"}

make clean

if [[ -n "${PETSC_ARCH:-}" ]]; then
    make mesh-phase1 \
        USE_MPI=1 \
        USE_PETSC=1 \
        PETSC_DIR="$PETSC_DIR" \
        PETSC_ARCH="$PETSC_ARCH" \
        MPI_CXX=mpicxx
else
    make mesh-phase1 \
        USE_MPI=1 \
        USE_PETSC=1 \
        PETSC_DIR="$PETSC_DIR" \
        MPI_CXX=mpicxx
fi

test -x "$CBS_ROOT/cbs3d_mesh_phase1"

echo
echo "Build successful:"
ls -lh "$CBS_ROOT/cbs3d_mesh_phase1"

```

E Phase 1 Slurm job

Listing 5: phase1_test.slurm

```

#!/bin/bash --login
#SBATCH --job-name=cbs3d_phase1
#SBATCH --output=/scratch/s.2337862/CBS3D++_SI/logs/phase1.%j.out
#SBATCH --error=/scratch/s.2337862/CBS3D++_SI/logs/phase1.%j.err
#SBATCH --partition=compute
#SBATCH --nodes=1
#SBATCH --ntasks=4

```

```

#SBATCH --cpus-per-task=1
#SBATCH --mem=64G
#SBATCH --time=00:30:00

set -euo pipefail

SCRATCH_ROOT="/scratch/s.2337862"
CBS_ROOT="${SCRATCH_ROOT}/CBS3D++_SI"
MODULE_FILE="${CBS_ROOT}/sunbird_modules.sh"
CASE_NAME="${CBS_CASE:-LidDrivenCavity3D}"

mkdir -p "$CBS_ROOT/logs"

# shellcheck source=/dev/null
source "$MODULE_FILE"

cd "$CBS_ROOT"

export OMP_NUM_THREADS=1
export OMP_PROC_BIND=close
export OMP_PLACES=cores

echo "Job ID:           ${SLURM_JOB_ID}"
echo "Node list:        ${SLURM_JOB_NODELIST}"
echo "Allocated tasks:   ${SLURM_NTASKS}"
echo "Case:              ${CASE_NAME}"
echo "PETSC_DIR:         ${PETSC_DIR}"
echo "PETSC_ARCH:        ${PETSC_ARCH:-unset}"
module list

test -x ./cbs3d_mesh_phase1

echo
echo "----- Test A: one rank, simple partitioner -----"
srun --ntasks=1 --cpus-per-task=1 \
    ./cbs3d_mesh_phase1 "$CASE_NAME" \
    -petscpartitioner_type simple

echo
echo "----- Test B: two ranks, simple partitioner -----"
srun --ntasks=2 --cpus-per-task=1 \
    ./cbs3d_mesh_phase1 "$CASE_NAME" \
    -petscpartitioner_type simple

echo
echo "----- Test C: four ranks, automatic partitioner -----"
srun --ntasks=4 --cpus-per-task=1 \
    ./cbs3d_mesh_phase1 "$CASE_NAME"

echo
echo "All Phase 1 Slurm test steps completed."

```

F Current serial/OpenMP Slurm job

Listing 6: serial_openmp.slurm

```

#!/bin/bash --login
#SBATCH --job-name=cbs3d_serial

```

```

#SBATCH --output=/scratch/s.2337862/CBS3D++_SI/logs/serial.%j.out
#SBATCH --error=/scratch/s.2337862/CBS3D++_SI/logs/serial.%j.err
#SBATCH --partition=compute
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=40
#SBATCH --mem=128G
#SBATCH --time=04:00:00

set -euo pipefail

SCRATCH_ROOT="/scratch/s.2337862"
CBS_ROOT="${SCRATCH_ROOT}/CBS3D++_SI"
MODULE_FILE="${CBS_ROOT}/sunbird_modules.sh"
CASE_NAME="${CBS_CASE:-LidDrivenCavity3D}"
EXECUTABLE="${CBS_EXECUTABLE:-cbs3dpp_si_petsc}"

mkdir -p "$CBS_ROOT/logs"

# shellcheck source=/dev/null
source "$MODULE_FILE"

cd "$CBS_ROOT"

export OMP_NUM_THREADS="${SLURM_CPUS_PER_TASK}"
export OMP_PROC_BIND=close
export OMP_PLACES=cores

echo "Job ID:           ${SLURM_JOB_ID}"
echo "Node list:        ${SLURM_JOB_NODELIST}"
echo "OpenMP threads:    ${OMP_NUM_THREADS}"
echo "Case:              ${CASE_NAME}"
echo "Executable:        ${EXECUTABLE}"
module list

test -x "./${EXECUTABLE}"

srun --ntasks=1 --cpus-per-task="${SLURM_CPUS_PER_TASK}" \
    "./${EXECUTABLE}" "$CASE_NAME"

```

G Conditional PETSc source-build script

Listing 7: build_petsc_fallback.sh

```

#!/bin/bash --login
set -euo pipefail

# Use only when Sunbird does not provide a compatible PETSc module.
# Run inside an allocated compute session, not as a heavy build on a login node.

SCRATCH_ROOT="/scratch/s.2337862"
SOURCE_ROOT="${SCRATCH_ROOT}/software/src"
INSTALL_ROOT="${SCRATCH_ROOT}/software/petsc-release"

mkdir -p "$SOURCE_ROOT"
cd "$SOURCE_ROOT"

if [[ ! -d petsc ]]; then

```

```
git clone --depth 1 --branch release https://gitlab.com/petsc/petsc.git
fi

cd petsc

./configure \
  --prefix="$INSTALL_ROOT" \
  --with-cc=mpicc \
  --with-cxx=mpicxx \
  --with-fc=mpif90 \
  --with-debugging=0 \
  COPTFLAGS="-O3" \
  CXXOPTFLAGS="-O3" \
  FOPTFLAGS="-O3" \
  --download-fblaslapack \
  --download-hypre \
  --download-metis \
  --download-parmetis \
  --download-ptscotch

make all
make check
make install

echo
echo "Add these lines to sunbird_modules.sh after the compiler and MPI modules:"
echo "export PETSC_DIR=$INSTALL_ROOT"
echo "unset PETSC_ARCH"
```

H Source register

1. Supercomputing Wales, “About Sunbird,” current system overview ([official Sunbird overview](#)).
2. Supercomputing Wales tutorial, “Running Jobs with Slurm” ([official Slurm tutorial](#)).
3. Supercomputing Wales User Guide, environment modules, Slurm, development environment and storage ([official user guide PDF](#)).
4. Supercomputing Wales rapid-start notes, including scratch retention ([official rapid-start page](#)).
5. PETSc installation and external-package guidance ([PETSc installation guide](#)).
6. PETSc supported external software ([PETSc external-software guide](#)).
7. PETSc DMPlex cell-list construction ([DMPlexCreateFromCellListPetsc reference](#)).
8. PETSc DMPlex distribution ([DMPlexDistribute reference](#)).